

ALGM Staking (DEFI)

SMART CONTRACT

Security Audit

Performed on Contracts:

LiquidStaking.sol
LiquidStakingAdmin.sol
LiquidStakingMain.sol
LiquidStakingManager.sol
LiquidStakingStorage.sol
LiquidStakingVoting.sol

Platform

ASTR

hashlock.com.au

JUNE 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Audit Findings	13
Centralisation	32
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The ALGM Staking partnered with Hashlock to conduct a security audit of their ALGM Staking smart contracts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

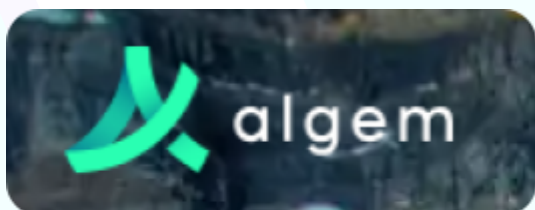
ALGM Staking is a DeFi dApp built on ASTR Network that allows you to stake/unstake ASTR Tokens to get xnASTR Tokens and interact with the protocol to fully unlock the potential of ALGM Staking.

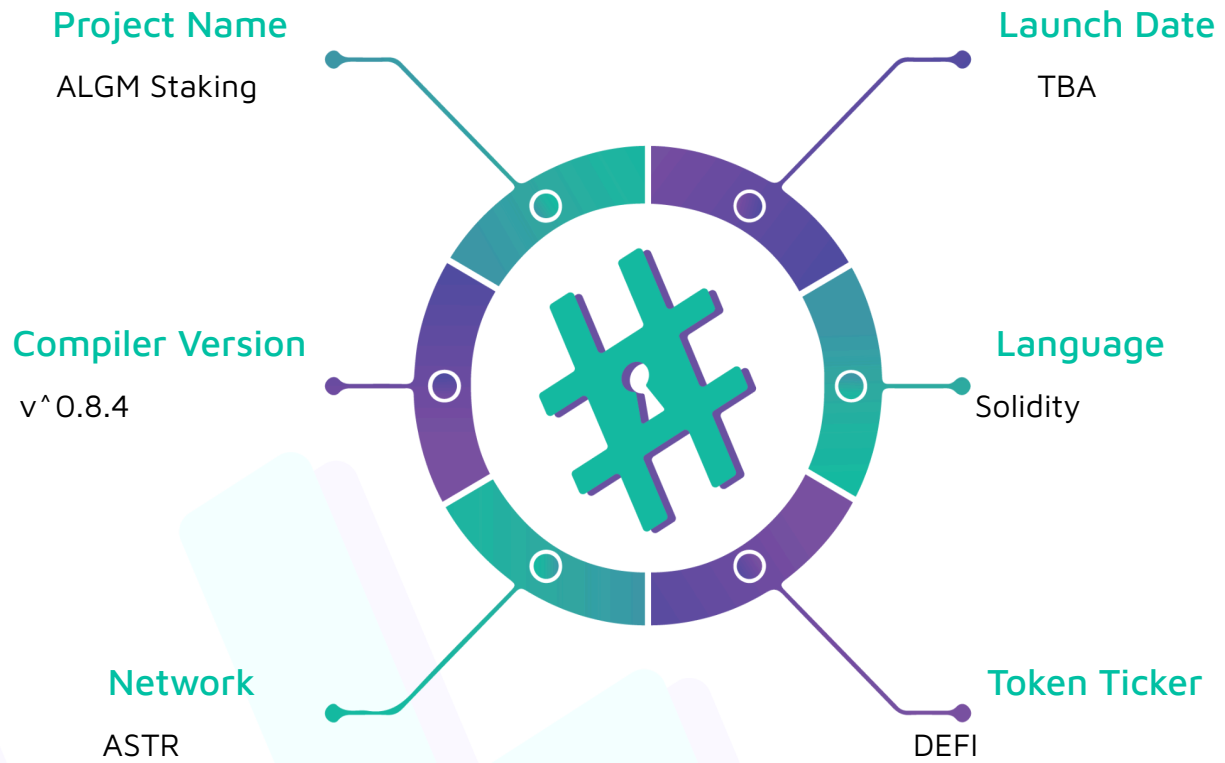
Project Name: ALGM Staking

Compiler Version: ^0.8.4

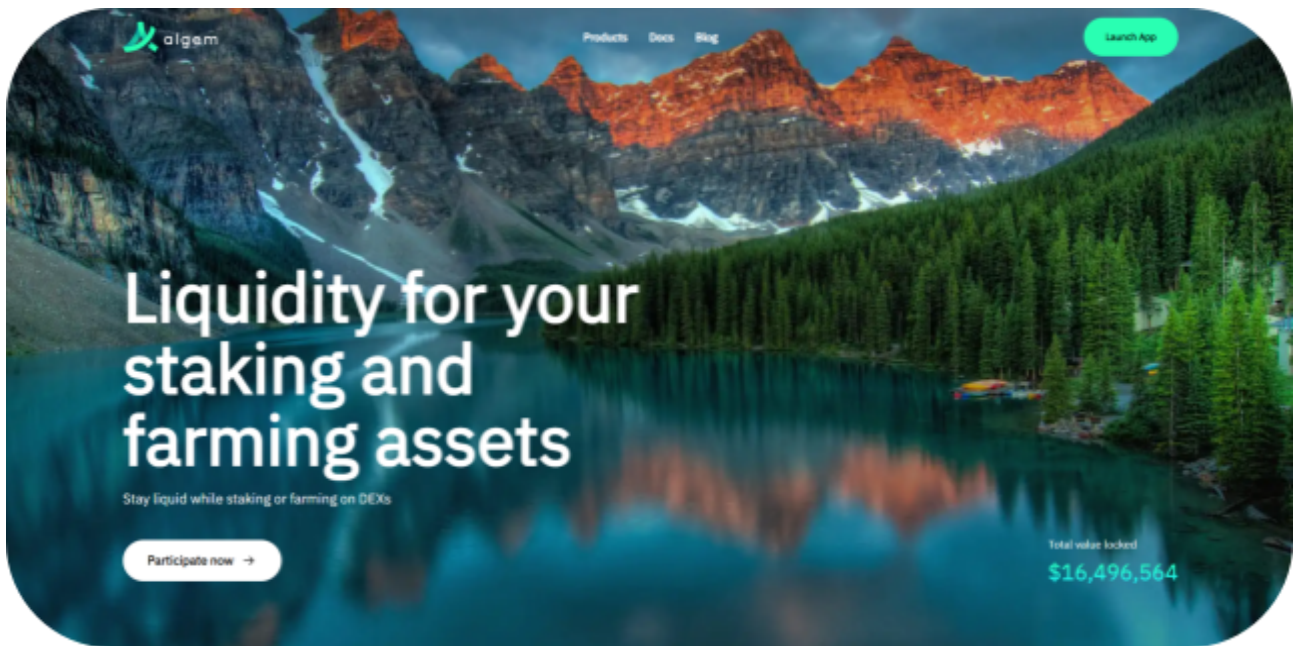
Website: www.algem.io

Logo:

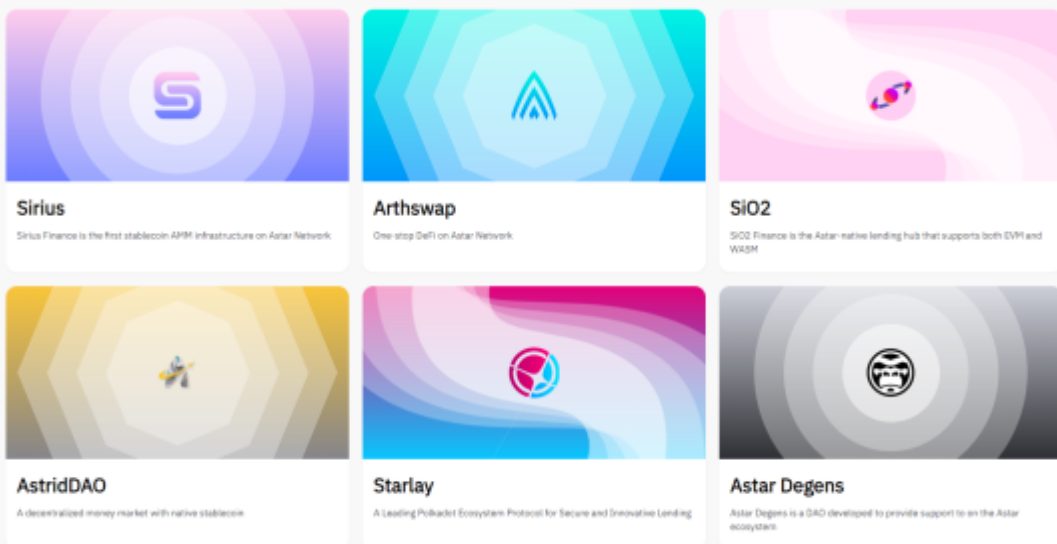


Visualised Context:

Project Visuals:



Algem ecosystem partners



#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the solidity code within the ALGM Staking project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	ALGM Staking Smart Contracts
Platform	ASTR / Solidity
Audit Date	June, 2024
Contract 1	LiquidStaking.sol
Contract 1 MD5 Hash	037e461be3afed7914476b59b23eb8ca
Contract 2	LiquidStakingAdmin.sol
Contract 2 MD5 Hash	f376b2cc8a112869fa7fccd51392fcb4
Contract 3	LiquidStakingMain.sol
Contract 3 MD5 Hash	db6a7992b063e4c3e654e0701dd46f3d
Contract 4	LiquidStakingManager.sol
Contract 4 MD5 Hash	99227629fd685575713724973baf8ae5
Contract 5	LiquidStakingStorage.sol
Contract 5 MD5 Hash	15c9740d5e23bbe089371244348b4d47
Contract 6	LiquidStakingVoting.sol
Contract 6 MD5 Hash	912043e533d2fc56499013b7658f07fc

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

2 High-severity vulnerabilities

1 Medium-severity vulnerabilities

2 Low-severity vulnerabilities

2 QA Optimisations

2 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
LiquidStakingMain.sol - Allows users to: - Stake - Unstake - stakeNFT & xnASTR Tokens - Claim Rewards - Withdraw - unstake	Contract achieves this functionality.
LiquidStakingAdmin.sol - Allows Admin to: - addNFT - RestakeFromPool - WithdrawBonusRewards - Set Min Stake Amount - Withdraw Revenue	Contract achieves this functionality.
LiquidStakingManager.sol - Allows Admin to: - Add Selectors - Add Selectors in batch - Delete Selector - Change Selectors - Add Manager - Remove Manager	Contract achieves this functionality.
LiquidStakingStorage.sol - Store different infos about contracts	Contract achieves this functionality.

LiquidStakingVoting.sol - Allows Admin to: - Vote - Unvote - Add Dapp - Set Default Weights	Contract achieves this functionality.
---	--

Code Quality

This Audit scope involves the smart contracts of the ALGM Staking, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the ALGM Staking smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] LiquidStakingMain#unstake - Insolvency when multiple users stake and unstake tokens.

Description

When multiple users stake and then unstake tokens, not all of them will receive tokens back.

This is because there is a logic error in the unstake function that leads to wrong calculations of rewards.

Vulnerability Details

In unstake() function there are 2 ways to unstake tokens: immediate unstake and unstake via Withdrawal

Unstake Via Withdrawal() :

When a User stakes X Amount of tokens, after some time can withdraw that, receiving the rewards generated from the stake.

To do this, when the user calls the unstake() function, there are calls on different functions via the updateAll() modifier.

These functions update the era, claimTokens from DAPPS_Staking...

Once all these functions are called, there is interaction with the unstake() function.

```

function unstake(uint256 _xnastrAmount, bool _immediate) external updateAll {

    if (_xnastrAmount == 0) revert ZeroAmountUnstake();

    if (xnASTR.balanceOf(msg.sender) < _xnastrAmount) revert
NotEnoughTokenBalance();

    uint256 astrAmount = getASTRValue(_xnastrAmount);

    _burn(msg.sender, _xnastrAmount);

    . . .

    if (astrAmount > totalStaked) {

        uint256 remains = astrAmount - totalStaked;

        rewardPool -= remains;

        astrAmount = totalStaked;

    }

    . . .

}

```

This snippet of code show initially, how the unstake() function works when is called from a user and it's not immediate.

Following the workflow of this function:

- 1) Check if the _xnastrAmount is == to 0 and if it is revert.
- 2) Check if the msg.sender xnASTR Balance is > than the _xnastrAmount passed as a parameter of the function.
- 3) convert the value of xnASTR Tokens to get ASTR Amount
- 4) Burn xnASTR Tokens

At this point, there is an error.

5) The if condition will check if the astrAmount, coming from the xnASTR conversion, is bigger than the totalStaked.

6) If it is bigger, check the difference between the astrAmount and the totalStaked (remains).

7) Decrease the rewardPool Amount

8) astrAmount = totalStaked

Now what is missing here is to decrease the totalStaked to 0, and to send the ASTR amount back to the user.

TotalStaked must be decreased to 0 otherwise the amount subtracted from every DAPPS_STAKING will be wrong.

At the same time, no tokens are sent to the user.

Proof of Concept

An example of a test that simulates this bug.

```
function test_stakeUnstakeMultiple()public{  
  
    vm.prank(deployer);  
  
    liquidMain.stake{value:1 ether}();  
  
  
    vm.prank(user);  
  
    liquidMain.stake{value:10 ether}();  
  
  
    vm.prank(user3);
```

```
liquidMain.stake{value: 8 ether}();

vm.prank(user2);

liquidMain.stake{value: 4 ether}();

vm.roll(100000);

uint xNastrBalanceUser = xnastr.balanceOf(user);

uint xNastrBalanceUser2 = xnastr.balanceOf(user2);

uint xNastrBalanceUser3 = xnastr.balanceOf(user3);

vm.prank(user);

liquidMain.unstake(xNastrBalanceUser, false);

vm.prank(user3);

liquidMain.unstake(xNastrBalanceUser3, false);

vm.prank(user2);

liquidMain.unstake(xNastrBalanceUser2, false);

vm.roll(200000);

uint currentERA = liquidMain.currentEra();

vm.prank(deployer);

liquidMain.sync(currentERA);

vm.roll(300000);

vm.prank(user);
```

```
liquidMain.withdraw(0);  
  
vm.prank(user2);  
liquidMain.withdraw(0);  
  
vm.prank(user3);  
liquidMain.withdraw(0);  
  
}
```

Impact

This double missing leads the protocol to be insolvent because, at a certain point, the protocol doesn't have enough ASTR to pay back the unstake() for all the users.

Recommendation

In the `if (astrAmount > totalStaked)` condition, update the totalStaked variable and send back tokens to the user.

Status

Resolved

[H-02] LiquidStakingMain# releaseCashBack - There is a logic error that leads to the loss of NFT and xnASTR Tokens for users.

Description

There is a logic error in this function that will lead to a loss of NFT and xnASTR Tokens for a user who interacts with the protocol.

Vulnerability Details

To receive ASTR rewards, a user that holds a NFT(accepted from the protocol), can call the addCashbackLock() function to send X amount of xnASTR Tokens plus the NFT to the liquidStakingMain contract.

After that, if the user wants to get back NFT and the xnASTR amount locked, will call the releaseCashbackLock() function.

```
function releaseCashbackLock(address _nftAddr, uint256 _amount) external {  
  
    IAlgemNFT nftContract = IAlgemNFT(_nftAddr);  
  
    Nft storage nft = nfts[_nftAddr];  
  
    CashbackLock storage lock = cashbackLocks[msg.sender][_nftAddr];  
  
  
    if (!nft.isActive) revert WrongNFTRelease();  
  
    if (lock.amount == 0) revert NoCashbackLocks();  
  
    if (_amount > lock.amount) revert NotEnoughLockedALGM();  
  
    collectedCashback[msg.sender] += lock.amount * nft.arps / REWARDS_PRECISION -  
    lock.debt;  
  
  
    if (_amount == lock.amount) {
```

```

        delete cashbackLocks[msg.sender][_nftAddr];

        nftContract.transferFrom(address(this), msg.sender, lock.tokenId);

    } else {

        . . .

    }

    . . .

}

```

The workflow of this function is simple:

- 1) check the ALGM NFT
- 2) Get infos from nft
- 3) check info about cashBackLock from msg.sender
- 4) Make some checks
- 5) Increment collectedCashBack

At this point, if the `_amount` the user wants to get back is equal, to the amount the user locked via the `addCashBackLock()` function, this function should send back the NFT to the user, but it doesn't.

Suppose there is a user that has minted the NFT With Token ID 1 and wants to interact with the protocol.

First of all the user will stake some ASTR to get `xnASTR` and after that will call the `addCashBackLock()` function passing the total amount of `xnASTR` token and the nft with the TokenId 1.

At this point the `cashbackLocks[msg.sender][_nftAddr]` is updated with the parameter passed to the function.

When the same user calls the `releaseCashbackLock()` function and goes into the if condition, the first thing this function does is:

```
delete cashbackLocks[msg.sender][_nftAddr].
```

At this moment, all the variables of the struct for the user are set to 0, and when the function reaches

```
nftContract.transferFrom(address(this), msg.sender, lock.tokenId);
```

It can't send back the token to the `msg.sender` because the `lock.tokenId` is equal to 0, and the `LiquidMain` only has the NFT with `tokenId 1`.

**** Tips**

In this case:

User A ==> NFT TokenId 0

User B ==> NFT TokenId 1

When user B calls the `releaseCashBack` function before then User A, will receive the NFT with TokenID 0 instead of the NFT TokenId 1.

When User A calls the `releaseCashBack` function before then User B, will receive the Token 0 back, and User B will not receive anything.

Proof of Concept

An example of a test that simulates this error:

```
function test_releaseCashbackGoneWrong()public{

    vm.prank(deployer);

    liquidMain.stake{value:1 ether}();

    vm.startPrank(user);

    liquidMain.stake{value:10 ether}();

    uint xnastrUserBalance = xnastr.balanceOf(user);

    nftContract.mint(user, 1);

    xnastr.approve(address(liquidMain), xnastrUserBalance);

    nftContract.approve((address(liquidMain)), 1);

    liquidMain.addCashbackLock(address(nftContract), xnastr.balanceOf(user)/2, 1);

    uint xnastrUserBalance2 = xnastr.balanceOf(user);

    vm.stopPrank();

    vm.roll(200000);

    uint currentERA2 = liquidMain.currentEra();

    vm.prank(deployer);

    liquidMain.sync(currentERA2);

    vm.roll(300000);
```

```
        (uint amountCashBack, ,) = liquidMain.cashbackLocks(user,  
address(nftContract));  
  
        vm.prank(user);  
  
        liquidMain.releaseCashbackLock(address(nftContract), amountCashBack);  
    }
```

Impact

Logic Error that leads to loss of NFT and ASTR Tokens for users.

At the same time nfts and ASTR will be locked in the protocol.

Recommendation

Fix the logic to send back NFTs to original owners correctly.

Status

Resolved

Medium

[M-01]LiquidStakingMain#addCashbackLock,releaseCashbackLock,claimCashback - Wrong calculations to set the cashbackLocks of a user on a specific NFT, lead to loss of rewards for the user.

Description

Wrong reward calculations when a user calls twice addCashbackLock() almost at the same time.

Calculations of the reward will not be updated the second time, and it's caused by the arps value in the NFT Struct.

Vulnerability Details

If a user tries to addCashbackLock() two times, the calculations for rewards are not correctly updated.

```
function addCashbackLock(address _nftAddr, uint256 _amount, uint256 _tokenId) external
{
    . . .

    CashbackLock storage lock = cashbackLocks[msg.sender][_nftAddr];

    if (lock.amount == 0) {

        lock.tokenId = _tokenId;

        nftContract.transferFrom(msg.sender, address(this), _tokenId);

    } else {

        collectedCashback[msg.sender] += lock.amount * nft.arps / REWARDS_PRECISION -
lock.debt;

    }
}
```

```
}

```

When a user calls this function, the first time, a new lock is created.

If a user tries to stake straight away again, it will go to the else statement of this function.

At this point `collectedCashback[msg.sender]` it's calculated using some parameters like: `lock.amount`, `nft.arps`, `REWARD_Precision` and `lock.debt`.

In this specific case, the result of this multiplication will be 0.

This happens because the `nft.arps` will be updated only when these are collected.

This means that to update that value the only way we have, is to call the `sync()` function (ADMIN Side) or have a kind of interaction with the protocol (stake, unstake....) using the `updateAll()` modifier.

Proof of Concept

An example of a test that simulates this error:

```
function test_addCashback()public{

    vm.prank(deployer);

    liquidMain.stake{value:1 ether}();

    vm.startPrank(user);

    liquidMain.stake{value:10 ether}();

    uint xnastrUserBalance = xnastr.balanceOf(user);
}
```

```

nftContract.mint(user, 1);

xnastr.approve(address(liquidMain), xnastrUserBalance);

nftContract.approve((address(liquidMain)), 1);


// First addCashBack

liquidMain.addCashbackLock(address(nftContract), xnastrUserBalance/3, 1);

console.log("cashBack Amount", liquidMain.collectedCashback(user));


// Second addCashBack

liquidMain.addCashbackLock(address(nftContract), xnastrUserBalance/3, 1);

console.log("cashBack Amount", liquidMain.collectedCashback(user));

vm.stopPrank();


vm.roll(200000);

vm.startPrank(deployer);

liquidMain.sync(liquidMain.currentEra());

vm.stopPrank();

vm.roll(300000);


//// Third addCashBack

//vm.startPrank(user);

//liquidMain.addCashbackLock(address(nftContract), xnastrUserBalance/3, 1);

//console.log("cashBack Amount", liquidMain.collectedCashback(user));

//vm.stopPrank();


//vm.roll(400000);

```

```

//

//vm.startPrank(deployer);

//liquidMain.sync(liquidMain.currentEra());

//vm.stopPrank();

//

//vm.roll(5000000);

        (uint lockAmountFromUser, ) = liquidMain.cashbackLocks(user,
address(nftContract));

    console.log(lockAmountFromUser);

    vm.startPrank(user);

    liquidMain.releaseCashbackLock(address(nftContract), lockAmountFromUser);

    console.log("cashBack Amount", liquidMain.collectedCashback(user));

    address[] memory path = new address[](1);

    path[0] = address(nftContract);

    console.log("UserBalance Before claim CashBack", address(user).balance);

    liquidMain.claimCashback(path);

    console.log("UserBalance After claim CashBack", address(user).balance);

}

```

Note

Uncomment the commented code to have a better check of the scenario.

Impact

Wrong reward amount for users

Recommendation

Find a different way to calculate rewards.

Status

Resolved

Low

[L-01] LiquidStakingMain# addCashbackLock- A logic error that leads to being impossible to addCashBackLock more than one time for the same user.

Description

Users should be able to call addCashbackLock() every time they want, but in this moment, if the users don't have NFTs for each call they can't.

Logic of this bug is:

The first time a user calls the addCashBackLock() function, will send X amount of ASTR Tokens and the NFT.

```
function addCashbackLock(address _nftAddr, uint256 _amount, uint256 _tokenId) external
{
    IAlgemNFT nftContract = IAlgemNFT(_nftAddr);

    Nft storage nft = nfts[_nftAddr];

    if (!nft.isActive) revert WrongNFTAdding();

    if (xnASTR.balanceOf(msg.sender) <= _amount) revert NotEnoughXNASTRForLock();
```

```

        if (nftContract.balanceOf(msg.sender) == 0) revert NotEnoughNFTForLock();

        . . .

    }

```

At this moment when the user calls again this function, at a certain point, will reach an if condition:

```

if (nftContract.balanceOf(msg.sender) == 0) revert NotEnoughNFTForLock();

```

Now if the user has sent already the NFT (First call), don't need to send a new one to increase the locked amount.

Recommendation

Change the logic to give the user the possibility to call more than once the addCashbackLock() function.

Status

Resolved

[L-02] LiquidStakingMain# setAlgStakingShare - There is the possibility to set up to 100% of the ALGM Staking Shares.

Description

There is the possibility to set the algStakingShare variable, up to 100%, via the setAlgStakingShare function.

This variable is involved in different calculations.

Recommendation

Add a requirement that checks the value passed at this function.

This is because there will not be the possibility to set an "out of range" value.

Status

Acknowledged

QA

[QA-01] LiquidStakingAdmin# partiallyPause, partiallyUnpause- These functions should be deleted, because there are no interactions in any contracts.

Recommendation

Delete these functions

Status

Resolved

[QA-02] LiquidStakingVoting# availableVotingPower- Use `>=` instead `>` to compare values.

Description

The most important thing is that the balance of veALGM of the caller is not `<` than total Votes used by the caller.

At this point, could be better use `>=` instead `>` .

```
function _availableVotingPower(address _user) internal view returns (uint256
votingPower) {
    if (veALGM.balanceOf(_user) - userVotes[_user].totalUsed >= 0)
        votingPower = veALGM.balanceOf(_user) - userVotes[_user].totalUsed;
}
```

Recommendation

Use `>=` instead `>`

Status

Resolved

Gas

[G-01] LiquidStakingAdmin#withdrawBonusRewards - `bonusRewardPool` storage variable is read multiple times

Description

The `bonusRewardPool` variable is read multiple times from storage.

Recommendation

Add a local variable instead of reading a storage variable multiple times.

Status

Resolved

[G-02] LiquidStakingVoting#_setDefaultWeights - `dappsList.length` is read in each iteration of for loop

Description

The `dappsList.length` variable is read in each iteration of for loop from storage.

Recommendation

Add a local variable for `dappsList.length`.

Status

Resolved

Centralisation

The ALGM Staking values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the ALGM Staking seems to have a sound and well-tested code base now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd